

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Тольяттинский государственный университет

Кафедра: _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

(бакалаврская работа)

на тему:

**«Разработка настольного приложения статистического
прогнозирования исходов футбольных матчей»**

Студент: _____ / _____ /

Группа: _____

Руководитель: _____ / _____ /

Тольятти 2026

Аннотация

Тема работы: «Разработка настольного приложения статистического прогнозирования исходов футбольных матчей».

Работа посвящена проектированию и реализации настольного программного средства, которое по открытым протоколам и календарю соревнований оценивает вероятности исхода футбольного матча в формате 1 / X / 2 и показывает предварительный точный счёт как наиболее вероятную клетку распределения голов. Приложение рассчитано на персональный компьютер под управлением Windows 10/11 (64-bit) или macOS 12 и новее и поставляется как собранное настольное приложение (.exe / .app). Оно не является букмекерским сервисом: прогноз носит статистический характер и не является советом совершать ставки.

В теоретической части рассмотрены особенности прогноза футбольного матча, наивная эвристика «пять последних матчей и округление», рейтинг Эло, независимая пуассоновская модель голов по мотивам работ Махера, а также роль смеси двух оценок. Проведён обзор существующих решений спортивной аналитики и сформулированы требования к программному обеспечению с учётом лимитов бесплатного тарифа football-data.org и этических ограничений.

В проектной и практической частях описана слоистая архитектура приложения на языке Python с графическим интерфейсом на базе Flet, локальный кэш SQLite с ограниченным временем жизни записей, модуль расчёта вероятностей и экраны пользователя. Языковая модель используется только для краткого текстового пояснения уже вычисленных вероятностей и фактов карточки и не выбирает исход матча.

Результатом работы является работоспособное настольное приложение и пояснительная записка с приложениями: техническим заданием, описанием схемы кэша, руководством пользователя, листингами ключевых фрагментов и примерами прогнозов.

Ключевые слова: статистический прогноз, исход 1 / X / 2, рейтинг Эло, распределение Пуассона, настольное приложение, Flet, SQLite, football-data.org.

Оглавление

Аннотация	2
Введение	5
1 Предметная область и постановка задачи	8
1.1 Особенности прогноза футбольного матча и методы, принятые в работе	8
1.2 Существующие решения спортивной аналитики	16
1.3 Требования к программному обеспечению	18
2 Проектирование	21
2.1 Технологии и архитектура	21
2.2 Информационная модель и модуль прогноза	25
2.3 Пользовательский интерфейс	31
3 Реализация и оценка	35
3.1 Загрузка данных и прогноз	35
3.2 Графический интерфейс и пояснение	37
3.3 Тестирование и качество прогноза	39
Заключение	47
Список используемой литературы и используемых источников	49
Приложение А Техническое задание на разработку настольного приложения статистического прогнозирования исходов футбольных матчей	51
Приложение Б Описание локального кэша SQLite	54
Приложение В Руководство пользователя	57
Приложение Г Листинги ключевых фрагментов	59
Приложение Д Примеры прогнозов	61

Введение

Футбольный матч удобно описывать тремя несовместными исходами: победа хозяев поля, ничья и победа гостей. В спортивной аналитике эту тройку принято обозначать как 1 / X / 2. Точный счёт относится к другой задаче: клеток вида «2:1», «1:1», «0:0» много, и даже у явного фаворита наиболее правдоподобная из них редко набирает более десяти–пятнадцати процентов вероятности. Поэтому в прикладных системах важно не смешивать оценку распределения исхода с обещанием «так и будет сыграно».

Интерес к количественной оценке силы команд и вероятностей исхода существует давно. Классический рейтинг Эло, первоначально предложенный для шахмат, позднее применялся и к командным видам спорта [5], [9]. Для голов в ассоциативном футболе широко известны пуассоновские модели, восходящие к работе Махера [11], и их развитие с поправкой на зависимость низких счетов у Диксона и Коулза [4]. Открытые веб-сервисы и мобильные приложения показывают таблицы, форму и иногда собственные индексы силы, однако пользователь, которому нужна воспроизводимая локальная модель с явными формулами и без подмены расчёта языковой моделью, часто остаётся без прозрачного настольного инструмента.

Актуальность темы связана с тремя обстоятельствами. Во-первых, открытый REST API football-data.org версии 4 даёт календарь, статусы и итоговые счета для ограниченного набора соревнований бесплатного тарифа, что позволяет строить учебное и исследовательское приложение без коммерческого доступа к закрытым поставщикам данных [7]. Во-вторых, настольный интерфейс на кроссплатформенном наборе средств Flet позволяет оформить сценарий «лига – календарь – карточка прогноза» в одном окне без развёртывания веб-сервера [6]. В-третьих, в учебном и инженерном контексте важно зафиксировать границу между фактом протокола, оценкой алгоритма и выводом модели, а также явно отделить статистический прогноз от советов по ставкам.

Объект исследования – процесс получения и представления статистической оценки исхода футбольного матча на основе открытых исторических результатов и календаря. Предмет исследования – методы и программные средства настольного приложения, реализующего локальный расчёт вероятностей $1 / X / 2$ и предварительного точного счёта с кэшированием сетевых ответов.

Цель работы – разработать настольное приложение статистического прогнозирования исходов футбольных матчей, в котором вероятности $1 / X / 2$ и предварительный счёт считаются локальной моделью, а текстовое пояснение языковой модели только пересказывает уже полученные числа и факты.

Для достижения цели поставлены следующие задачи:

- проанализировать особенности прогноза футбольного матча и выбрать методы расчёта, пригодные для реализации в прикладном программном средстве;
- сопоставить существующие решения спортивной аналитики с требованиями к настольному приложению с локальной моделью;
- сформулировать функциональные и нефункциональные требования, включая этические ограничения и способ поставки готового настольного приложения;
- спроектировать архитектуру, информационную модель, модуль прогноза и пользовательский интерфейс;
- реализовать загрузку данных, кэш, расчёт и экраны приложения;
- провести модульное тестирование и описать метрики качества прогноза без выдуманных показателей «точности на чемпионате».

Методы исследования включают анализ научной и технической литературы по рейтингам и моделям голов, изучение официальной документации прикладных библиотек и API, проектирование послойной архитектуры, реализацию на языке Python и проверку свойств модели модульными тестами.

Практическая значимость работы состоит в готовом настольном приложении и комплекте пояснительной записки с техническим заданием, описанием локального кэша, руководством пользователя и примерами прогнозов. Научно-методическая значимость – в согласованном изложении цепочки «факт – оценка – вывод» и в явном отказе подменять локальную смесь Эло и Пуассона наивной эвристикой округления или ответом языковой модели. Научная новизна на уровне бакалаврской ВКР носит прикладной характер: согласованная реализация известного класса моделей (Эло и независимый Пуассон) в настольном приложении с явной границей ответственности языковой модели.

Структура пояснительной записки. Во введении обоснованы актуальность, цель и задачи. В главе 1 рассмотрены предметная область, методы, аналоги и требования. В главе 2 описаны технологии, архитектура, информационная модель, модуль прогноза и интерфейс. В главе 3 изложены реализация загрузки данных, графический интерфейс, тестирование и подход к оценке качества. В заключении приведены итоги и направления развития. В приложениях А–Д помещены техническое задание, схема кэша, руководство пользователя, листинги и примеры прогнозов.

Ограничения работы. Источником оперативных данных служит бесплатный тариф football-data.org: двенадцать кодов соревнований, не более десяти запросов в минуту, возможная задержка счёта и расписания [7]. Поля ожидаемых голов (xG), травм и гарантированного стартового состава в используемом контуре не доступны и в модель не домысливаются. Полная поправка Диксона–Коулза в текущей версии не реализована. Ретроспектива качества на полном чемпионате не заявляется; в главе 3 приведены определения метрик и учебный пример на локальной выборке.

Прогноз, который формирует приложение, является статистической оценкой. Он не гарантирует исход матча и не является советом ставить деньги.

1 Предметная область и постановка задачи

1.1 Особенности прогноза футбольного матча и методы, принятые в работе

Исход футбольного матча в прикладной аналитике чаще всего сводят к трём несовместным событиям: победа хозяев (1), ничья (X) и победа гостей (2). Такая тройка покрывает весь пространство элементарных исходов матча с точки зрения результата и удобна для отображения на экране: три доли, сумма которых равна единице. Точный счёт – событие другого рода. Число клеток «i:j» велико, а вероятность даже самой частой клетки у сильной команды обычно измеряется единицами или первыми десятками процентов. Поэтому в работе одновременно решаются две связанные, но разные задачи. Первая – оценить вероятности 1 / X / 2. Вторая – показать предварительный точный счёт как клетку с наибольшей вероятностью на сетке голов, не выдавая её за обещание конкретного результата.

В бытовой речи слово «прогноз» смешивает три разных слоя смысла. Без их разделения и текст, и программа начинают выдавать мнение за наблюдение.

Фактом в работе считается только то, что зафиксировано протоколом или официальным расписанием: кто с кем играет, когда, на чьём поле, каким счётом закончились уже сыгранные матчи той же лиги. Утверждения вида «команда в огне», «должен выиграть», «заслуживали 2:1» фактами не являются.

Оценкой называется величина, которую алгоритм выводит из фактов на дату расчёта: рейтинг силы, среднее число забитых и пропущенных за последние матчи. После нового тура оценка изменится. Это нормальное свойство скользящих показателей.

Выводом называется прогноз ещё не состоявшегося матча: три вероятности 1 / X / 2, ожидаемые голы, наиболее вероятный точный счёт. Вывод всегда сопровождается оговоркой: это распределение, а не приговор.

Источником фактов для оперативного контура служит открытый календарь и протоколы football-data.org (REST API версии 4) для двенадцати соревнований бесплатного тарифа [7]. Длинные исторические таблицы football-data.co.uk используются для ретроспективы и обучения, а не как подмена протокола в момент, когда пользователь открывает карточку матча [8]. Полей ожидаемых голов (xG), состава на игру и травм в бесплатном тарифе нет; работа их не домысливает.

Наивный приём «пять матчей и округление»

Распространённый бытовой способ «угадать счёт» выглядит так. У каждой команды берут пять последних состоявшихся матчей, считают среднее арифметическое забитых голов, затем округляют результат хозяев в большую сторону, а результат гостей – в меньшую. Пример: хозяева забили 2, 1, 0, 3 и 2 – среднее 1,6, вверх это 2; гости забили 1, 1, 2, 0 и 1 – среднее 1,0, вниз это 1; «точный счёт» объявляют 2:1.

В открытых русскоязычных схемах приём обычно дополняют пропущенными голами и сводят атаку одной стороны с обороной другой:

$$K_1 = \frac{G^{(заб)}_{(дом)} + G^{(np)}_{(гост)}}{2}, K_2 = \frac{G^{(заб)}_{(гост)} + G^{(np)}_{(дом)}}{2}, \quad (1)$$

где $G^{(заб)}_{(дом)}$ – среднее число голов, забитых хозяевами за последние пять матчей; $G^{(np)}_{(гост)}$ – среднее число голов, пропущенных гостями за последние пять матчей; $G^{(заб)}_{(гост)}$ – среднее число голов, забитых гостями; $G^{(np)}_{(дом)}$ – среднее число голов, пропущенных хозяевами.

Далее K_1 и K_2 превращают в целые числа округлением вверх у хозяев и вниз у гостей (или иным бытовым правилом). Полученная пара объявляется «точным счётом», а исход 1 / X / 2 выводят из сравнения целых.

У этого приёма есть педагогическая ценность: он показывает, что счёт можно вывести из истории голов, не привлекая слухи о составе. Но как основной метод он непригоден, и в работе это сказано прямо.

Во-первых, пять голов аутсайдеру не равны пяти голам лидеру: средние не чувствуют силу соперника. Во-вторых, пять «любых» матчей смешивают домашние и выездные встречи. В-третьих, выборка из пяти наблюдений шумная: один разгром 5:0 ломает среднее. В-четвёртых, округление вверх у хозяев и вниз у гостей выкидывает дробную часть и систематически толкает ответ к счетам 2:1 и 1:0 – даже когда модель ждала 1,1 и 1,0. В-пятых, точный счёт – редкое событие; объявить одну клетку «прогнозом» без её вероятности значит скрыть неопределённость. В-шестых, ничья почти не моделируется, пока оба округления случайно не совпадут.

Поэтому приём «пять матчей и округление» в работе остаётся базовой линией для сравнения, а не движком экрана прогноза. Экран не имеет права подменить распределение одной картинкой 2:1.

Рейтинг Эло

Чтобы учесть, кому забивали, а не только сколько, в работе используется рейтинг Эло – классический способ сравнивать силу соперников по последовательности результатов [5]. Каждая команда начинает с 1500 пунктов. Перед матчем считается ожидание успеха хозяев с поправкой на домашнее поле (+80 пунктов к рейтингу хозяев):

$$E_{\text{(дом)}} = (1)/(1 + 10^{((R_{\text{(гост)}}) - (R_{\text{(дом)}}) + 80)) / 400}), \quad (2)$$

где $E_{\text{(дом)}}$ – ожидаемая доля успеха хозяев в шкале «победа = 1, ничья = 0,5, поражение = 0»; $R_{\text{(дом)}}$ – текущий рейтинг хозяев; $R_{\text{(гост)}}$ – текущий рейтинг гостей; 80 – константа домашнего преимущества в пунктах рейтинга; 400 – масштаб логистической кривой Эло.

Зависимость ожидания от разницы рейтингов удобно иллюстрировать графиком (рисунок 1). При равных силах с учётом домашнего плюса хозяева имеют ожидание выше одной второй; при большом перевесе фаворита кривая насыщается.

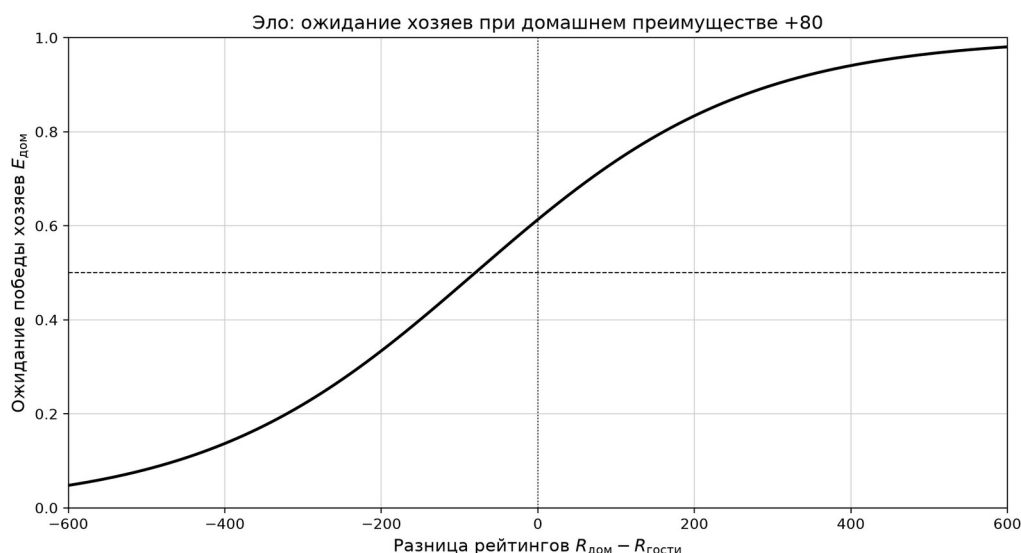


Рисунок 1 – Ожидание Эло от разницы рейтингов

Факт результата кодируется просто: победа – 1, ничья – 0,5, поражение – 0. Рейтинг сдвигается на двадцать пунктов, умноженных на разницу факта и ожидания:

$$R \rightarrow R + 20 \cdot (S - E), \quad (3)$$

где S – факт результата для обновляемой команды; E – её ожидание до матча; 20 – коэффициент чувствительности (К-фактор).

Сильный, который выиграл у слабого, почти не растёт; слабый, который взял очки у сильного, растёт заметно. Применение рейтинга Эло к футбольным матчам обсуждалось, в том числе, в работах по прогнозированию результатов ассоциативного футбола [9].

Из двух рейтингов можно получить и тройку 1 / X / 2: долю, которая остаётся после ничьей, делят пропорционально ожиданию Эло, а саму ничью задают гладкой функцией от разницы рейтингов. В принятой реализации

$$P(X) = \text{clip}(0,27 - 0,06 \cdot \tanh(|\Delta| / 200), 0,18, 0,32), \quad (4)$$

где $\Delta = R_{\text{дом}} + 80 - R_{\text{гост}}; \text{clip}(x, a, b)$ – ограничение значения отрезком $[a, b]$.

Вероятности победы хозяев и гостей:

$$P(1) = (1 - P(X)) \cdot E_{\text{дом}}, P(2) = (1 - P(X)) \cdot (1 - E_{\text{дом}}). \quad (5)$$

Вероятность ничьей лежит в диапазоне примерно от 18 % до 32 %: при равных силах она ближе к 27 %, при большой разнице снижается, но не обнуляется. Модель, которая «забывает» ничью, для футбола непригодна.

Эло опирается только на исходы уже сыгранных матчей той же лиги и только на те, что завершились до даты прогнозируемой встречи. Подглядывания в будущий счёт нет.

Распределение голов и независимый Пуассон

Эло хорошо разделяет фаворита и аутсайдера, но сам по себе не говорит, сколько мячей ждать. Для голов в работе принята классическая схема, восходящая к модели Махера: число мячей хозяев и гостей описывается распределениями Пуассона с интенсивностями $\lambda_{\text{дом}}$ и $\lambda_{\text{гост}}$ [11]. Интенсивность – это ожидаемое число голов, а не «точный счёт».

Сила атаки и обороны оценивается по средним забитым и пропущенным за последние пять матчей относительно типичного уровня лиги. В текущей реализации константа лиги задана как 1,35 гола на команду за игру (около 2,7 гола на матч). Относительные атаки и обороны:

$$a_{\text{дом}} = \max(0,4, G^{\text{заб}}_{\text{дом}} / 1,35), d_{\text{дом}} = \max(0,4, G^{\text{пр}}_{\text{дом}} / 1,35), \quad (6)$$

и аналогично для гостей. Пол в 0,4 не даёт обнулить интенсивность при короткой серии без голов.

Домашний фактор для ветки 1 / X / 2 входит через уже посчитанный рейтинг Эло. Пусть

$$\delta = (R_{\text{дом}} + 80 - R_{\text{гост}}) / 1000. \quad (7)$$

Тогда ожидаемые голы для смеси исходов:

$$\lambda_{\text{дом}} = \text{clip}(1,35 \cdot a_{\text{дом}} \cdot d_{\text{гост}} \cdot (1 + \delta), 0,25, 4), \quad (8)$$

$$\lambda_{\text{гост}} = \text{clip}(1,35 \cdot a_{\text{гост}} \cdot d_{\text{дом}} \cdot (1 - \delta), 0,25, 4). \quad (9)$$

Вероятность клетки (i, j) на сетке от 0:0 до 8:8 при независимости голов:

$$P(X = i, Y = j) = e^{-(\lambda_{\text{дом}})} \frac{\lambda_{\text{дом}}^i}{i!} \cdot e^{-(\lambda_{\text{гост}})} \frac{\lambda_{\text{гост}}^j}{j!}. \quad (10)$$

Сумма клеток выше диагонали даёт победу хозяев, на диагонали – ничью, ниже – победу гостей. Хвост за восемью голами отбрасывается, три вероятности затем нормируются так, чтобы в сумме давать единицу. Вид сетки для конкретного числового примера удобно показать графически (рисунок 2).

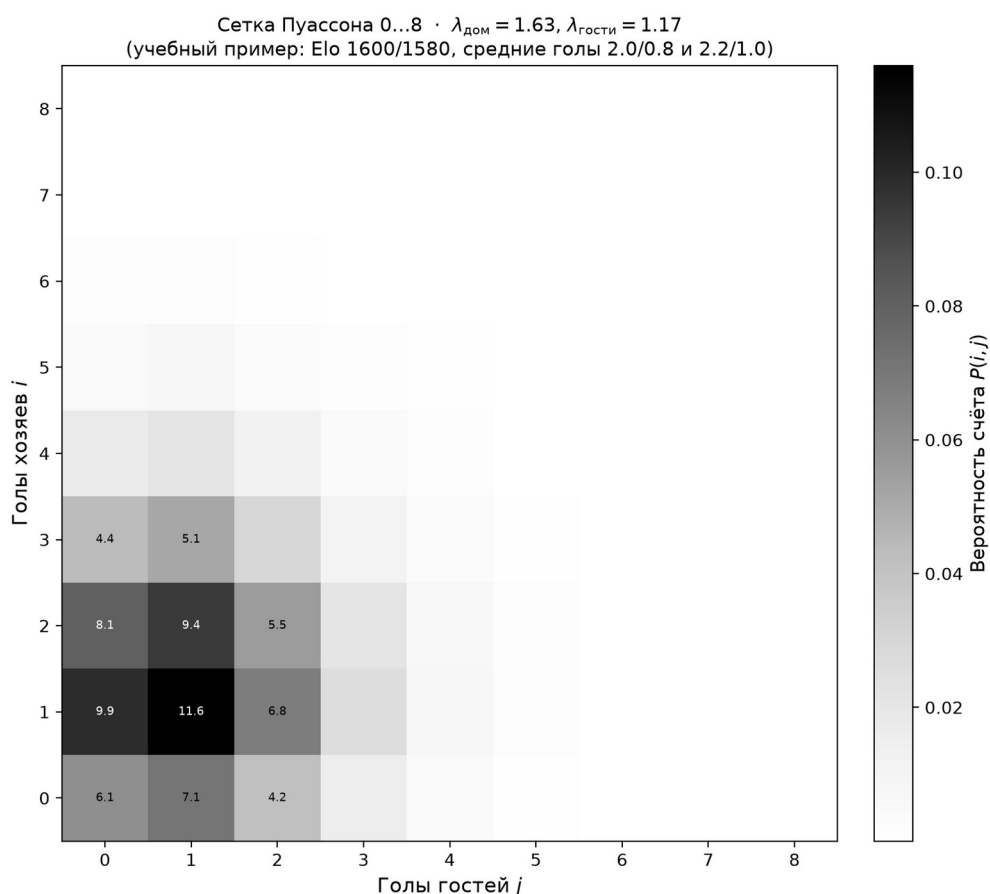


Рисунок 2 – Сетка вероятностей независимого Пуассона

Полная модель Диксона–Коулза с поправкой на зависимость низких счетов (0:0, 1:0, 0:1, 1:1) в текущей версии не используется [4]. Это

сознательное упрощение первой версии: независимость голов проще объясняется и проверяется. В обзоре литературы Диксон и Коулз остаются; в тексте реализации нельзя писать, будто поправка *rho* уже стоит в коде.

Смесь двух моделей для исхода 1 / X / 2

Итог на экране – не «чистый Эло» и не «чистый Пуассон». Если в кэше лиги меньше четырёх сыгранных матчей, сетка голов не запускается: выборка слишком мала, остаётся только Эло. Если матчей достаточно, тройка 1 / X / 2 считается дважды и смешивается:

$$P = 0,65 \cdot P_{\text{(Пуассон)}} + 0,35 \cdot P_{\text{(Эло)}}. \quad (11)$$

После смешивания вероятности снова приводятся к сумме 1. На одних и тех же исходных данных алгоритм даёт одни и те же три числа: расчёт детерминирован.

Почему смесь, а не одна модель? Пуассон чувствует, много ли забивают и пропускают в последнее время. Эло чувствует, против кого это происходило на длинной дистанции. Порознь каждая оценка слепа к одной стороне дела; вместе они закрывают бытовой соблазн «просто усреднить пять игр». Столбцы итоговых долей для учебного примера показаны на отдельном рисунке (рисунок 3).

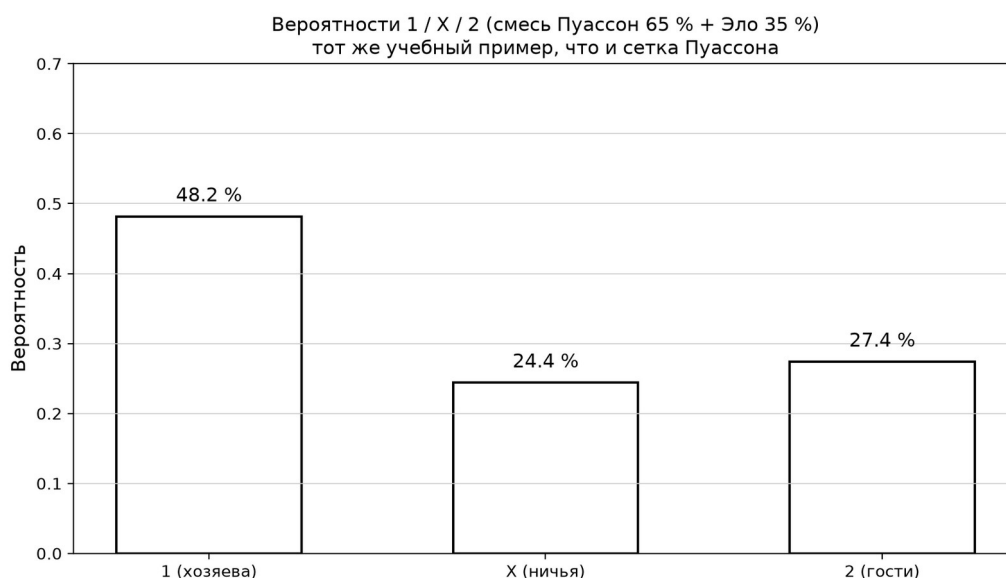


Рисунок 3 – Вероятности исходов 1 / X / 2

Языковая модель в этой цепочке не участвует. Она может только пересказать уже посчитанные вероятности и факты карточки двумя–четырьмя предложениями. Числа на экране не зависят от наличия пояснения.

Предварительный точный счёт без косвенных факторов

Отдельно от 1 / X / 2 пользователь вправе увидеть предварительный счёт, выведенный только из исторических голов. Сюда не входят рейтинг Эло, место в таблице, очные встречи как отдельный классификатор, травмы, состав, погода, мотивация и текст нейросети.

Ожидаемые голы для этой ветки совпадают по смыслу с K_1 и K_2 из формулы (1), но дальше округления нет: строится сетка Пуассона, выбирается клетка с наибольшей вероятностью. Если две клетки равны, предпочтение отдаётся той, что ближе к математическому ожиданию ($\lambda_{\text{дом}}$, $\lambda_{\text{гост}}$). На карточке рядом со счётом обязана стоять доля этой клетки и сами ожидаемые голы.

Так закрывается исходный вопрос постановки: да, предварительный счёт из истории посчитать можно; нет, его нельзя выдавать так, будто дробь 1,6 «естественно» становится двойкой у хозяев.

Числовой учебный пример

Пусть у хозяев рейтинг 1600, у гостей 1580; средние за пять матчей: хозяева забивают 2,0 и пропускают 0,8, гости забивают 2,2 и пропускают 1,0; в кэше лиги двадцать сыгранных матчей. Тогда относительные атаки и обороны примерно равны 1,48; 0,59; 1,63; 0,74, а $\delta = 0,10$. По формулам (8)–(9) получают $\lambda_{\text{дом}} \approx 1,63$, $\lambda_{\text{гост}} \approx 1,17$. После сетки и смеси (11) итоговые доли исхода (с округлением до четырёх знаков после запятой) составляют:

$$P(1) \approx 0,4817, P(X) \approx 0,2442, P(2) \approx 0,2741.$$

Для предварительного счёта Эло не участвует: $\lambda_{\text{дом}} = \lambda_{\text{гост}} = 1,50$, наиболее вероятная клетка – 1:1 с вероятностью около 0,112. Наивная эвристика с округлением тех же K_1 и K_2 дала бы 2:1. Модуль округлить 1,50 «в пользу хозяев» не имеет права.

Фактами в примере были только прошлые счета и расписание. Рейтинги, λ , три процента и клетка 1:1 – вывод.

Что пользователь видит и чего не видит

На карточке матча рядом оказываются величины разной природы. В расчёт $1 / X / 2$ входят рейтинг Эло и средние голы за пять матчей. В предварительный счёт входят только средние голы. Строка формы, краткая сводка очных встреч и строка турнирной таблицы – пояснение человеку. Они помогают читать карточку, но не являются вторым классификатором «кто победит».

Главный вывод по исходу – три процента, в сумме сто. Подпись «фаворит – хозяева» допустима лишь как ярлык на наибольшей из трёх долей. Предварительный счёт – ярлык на наибольшей клетке сетки, всегда с её вероятностью.

1.2 Существующие решения спортивной аналитики

Рынок спортивной информации насыщен веб-сайтами и мобильными приложениями. SofaScore и Flashscore ориентированы на live-счёт, статистику ударов, составы и уведомления. Их сильная сторона – оперативность и широта покрытия. Для учебной настольной системы с явной локальной моделью они служат скорее ориентиром по удобству календаря, чем образцом архитектуры расчёта: формулы силы команд у таких продуктов закрыты, а основной сценарий – потребление потока событий, а не воспроизводимый офлайн-расчёт.

Проекты в духе FiveThirtyEight / SPI публикуют рейтинги силы и вероятности исходов на основе собственных моделей. Они полезны как пример того, что вероятностный язык (а не один «верный счёт») стал нормой публичной аналитики. Вместе с тем пользователь не получает настольного приложения с исходным кодом своей модели и не контролирует кэш запросов к бесплатному API.

Understat и родственные ресурсы делают акцент на ожидаемых голах (xG) и визуализации ударов. Для настоящей работы xG недоступен в бесплатном тарифе football-data.org, поэтому сопоставление идёт на уровне идеи: более богатые признаки улучшают модель, но их нельзя выдумать, если поля нет в протоколе [7].

Общим контрастом для разрабатываемого приложения служат следующие свойства:

- настольное окно, а не обязательный браузерный клиент;
- локальный расчёт $1 / X / 2$ смесью Эло и независимого Пуассона с фиксированными константами первой версии;
- отказ от live-сканера счёта на бесплатном тарифе, где данные могут приходить с задержкой;
- языковая модель только как пояснение, не как классификатор исхода;
- явный дисклеймер: статистический прогноз, не совет ставить деньги.

Сводное сравнение удобно представить таблицей требований к прозрачности.

Таблица 1 – Сопоставление класса решений и свойств разрабатываемого приложения

Свойство	Типичный live-агрегатор	Публичный рейтинг-индекс	Разрабатываемое приложение
Настольное окно	редко	нет	да
Открытые формулы 1 / X / 2	нет	частично	да (Эло+Пуассон)
Локальный TTL-кэш под лимит API	не применимо	не применимо	да
xG в бесплатном контуре	часто есть	зависит от продукта	нет (поля нет в тарифе)
LLM как классификатор исхода	обычно нет	нет	запрещено
Дисклеймер «не совет ставить»	зависит от продукта	зависит от продукта	обязателен

Таким образом, аналоги задают планку удобства и показывают спектр данных, а проектируемое средство занимает нишу прозрачного учебного десктопа с ограниченным, но документированным контуром данных.

1.3 Требования к программному обеспечению

Требования сформулированы с опорой на практику оформления технического задания на программное средство и автоматизированную систему [1], [2]. Полный текст технического задания приведён в приложении А; ниже – сжатая постановка для основной части.

Функциональные требования:

- приложение должно предоставлять список соревнований бесплатного тарифа football-data.org (двенадцать кодов: PL, PD, SA, BL1, FL1, DED, PPL, ELC, BSA, CL, WC, EC);

- пользователь должен выбирать лигу и видеть календарь матчей с датой, названиями команд и статусом;

- по выбранному матчу система должна рассчитывать вероятности $1 / X$ / 2 локальной моделью и показывать предварительный точный счёт как моду сетки Пуассона по средним голам за пять матчей;

- карточка матча должна отображать поясняющие агрегаты: форму, рейтинги Эло, сводку очных встреч, места в таблице, размер кэша истории;

- допускается краткое текстовое пояснение уже посчитанных чисел языковой моделью; числа на экране не зависят от наличия пояснения;

- на экране должен присутствовать дисклеймер о статистическом характере прогноза.

Нефункциональные требования:

- платформы поставки: Windows 10/11 (64-bit) и macOS 12 и новее;

- способ запуска для пользователя: собранное настольное приложение (файл `.exe` на Windows, пакет `.app` на macOS), без обязательной установки интерпретатора Python конечным пользователем;

- язык реализации: Python 3.11 и выше [13];

- графический интерфейс: Flet [6];

- сетевой клиент: httpx с соблюдением лимита не более десяти запросов в минуту к football-data.org [7], [10];

- локальный кэш ответов: SQLite с временем жизни записей шесть часов для ещё не сыгранных матчей и сутки для завершённых матчей и таблиц [15];

- модульные тесты не обращаются к живому API и используют локальные образцы данных.

Этические и правовые ограничения:

- продукт не позиционируется как тотализатор и не обещает прибыль от ставок;

- использование данных подчиняется условиям поставщика football-data.org;

- языковая модель не имеет права выдумывать травмы, xG и иной счёт, отсутствующий в переданных фактах.

Критерии приёмки первой версии: собранное приложение запускается в настольном окне на целевой платформе; загружаются лиги и матчи; прогноз на фиксированном срезе фактов воспроизводим; сумма трёх вероятностей равна единице; предварительный счёт не зависит от Эло и не совпадает с наивным округлением вверх/вниз; пояснение LLM не подменяет числа.

Выводы по главе 1. Предметная область требует разделять факт, оценку и вывод и не подменять распределение исхода одной клеткой счёта. В качестве основного метода принята смесь рейтинга Эло и независимого Пуассона; эвристика пяти матчей остаётся базой сравнения. Обзор аналогов показал нишу прозрачного настольного приложения. Требования фиксируют состав функций, лимиты API, кэш SQLite, поставку .exe / .app, дисклеймер и запрет на выбор исхода языковой моделью. Домашнее преимущество входит в Эло сдвигом +80 и в ветку Пуассона через *delta*; в предварительный счёт – только косвенно через окно голов.

2 Проектирование

2.1 Технологии и архитектура

Выбор технологий подчинён цели: настольное окно, локальный расчёт, умеренный объём зависимостей и воспроизводимость на учебной машине. Язык Python 3.11 и выше даёт удобную работу с HTTP, таблицами и числовыми расчётами, а также зрелую экосистему тестирования [13]. Графический интерфейс строится на Flet: приложение оформляется как нативное окно на Windows и macOS, без обязательного развёртывания отдельного веб-сервера для пользователя [6]. Сетевые запросы выполняются библиотекой `httpx` [10]. Для разбора CSV истории применяется `pandas` [12]. Локальный кэш ответов API размещается в файле SQLite [15]. Библиотека `scikit-learn` входит в стек проекта как инфраструктура обучения и экспериментов на исторических CSV; оперативный контур экрана прогноза в первой версии опирается на явные формулы Эло и Пуассона, а не на непрозрачный классификатор «из коробки» [14].

Архитектура пакета намеренно слоистая. Слой пользовательского интерфейса обращается только к сервисам и к доменным типам. Слой сервисов оркестрирует загрузку данных, сбор признаков, вызов модели и при необходимости пояснение языковой моделью. Слой данных знает HTTP, CSV и SQLite, но не знает виджетов. Слой моделей считает вероятности и предварительный счёт, не выполняя сетевых запросов. Слой AI получает уже готовый JSON фактов и вероятностей и не выбирает исход. Доменные `dataclass` и перечисления не содержат ввода-вывода.

Схематично поток зависимостей выглядит так:

```
ui → services → data (football-data.org, CSV, SQLite)
                  → models (Эло, Пуассон)
                  → ai (пояснение)
ui → domain
```

Та же идея представлена на рисунке слоёв (рисунок 4). Контекст системы – пользователь, настольное приложение, внешний API, локальный кэш и опциональный сервис пояснения – показан на рисунке контекста (рисунок 5).

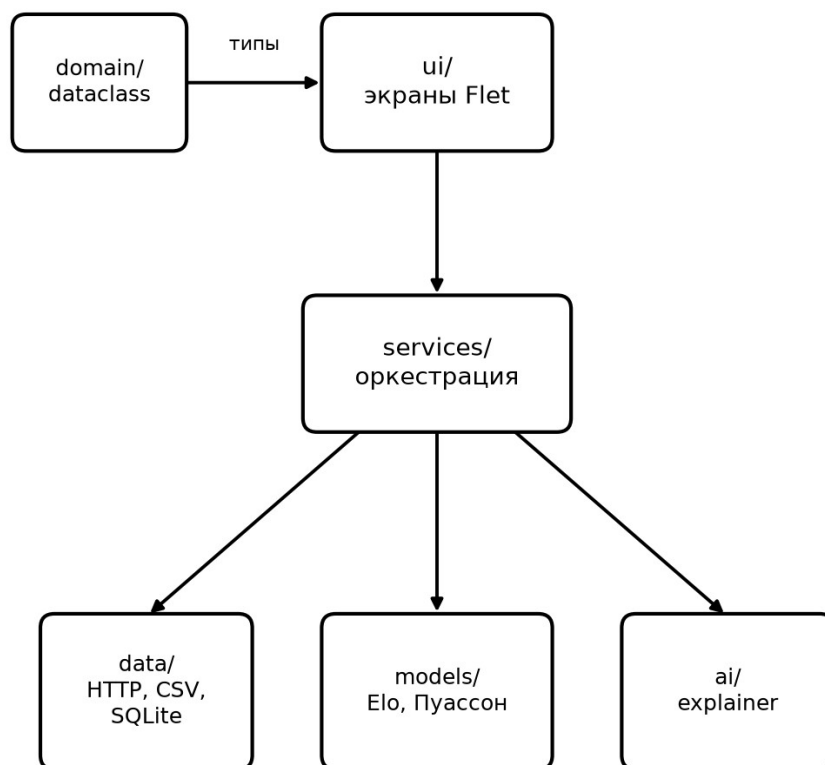


Рисунок 4 – Слоистая архитектура приложения

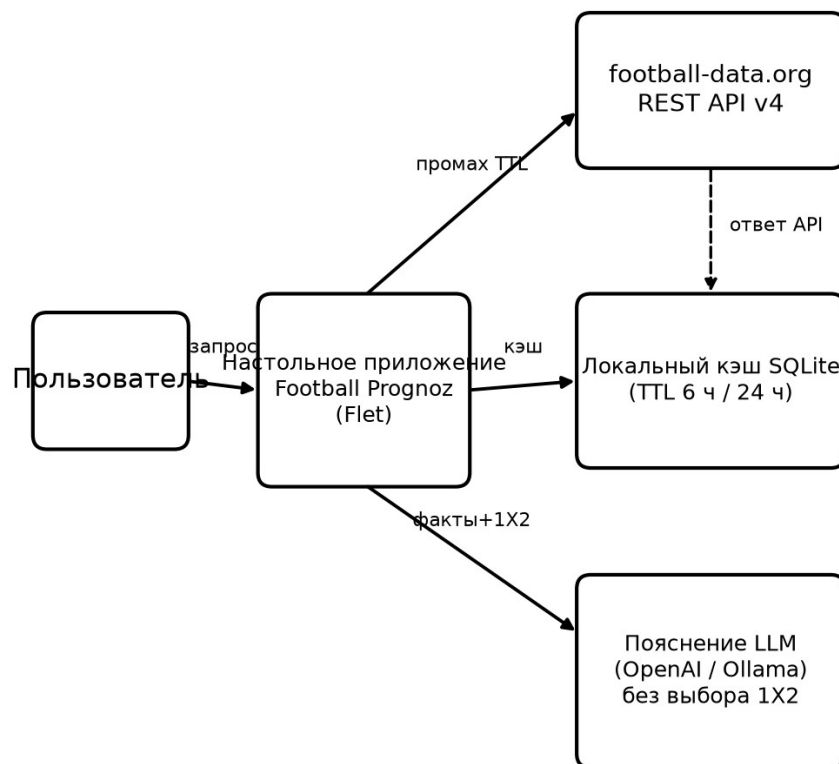


Рисунок 5 – Контекстная диаграмма системы

Последовательность расчёта прогноза:

1. Сервис матчей читает кэш SQLite; при промахе времени жизни записи обращается к football-data.org.
2. Сервис признаков собирает форму, очные встречи, рейтинги Эло, средние голы за пять матчей и места в таблице только из кэша, обрезая историю датой матча.
3. Модуль прогноза возвращает вероятности дома / ничья / гости (сумма равна 1): при малой выборке только Эло, иначе смесь Пуассона с весом 0,65 и Эло с весом 0,35. Предварительный счёт – мода сетки по средним голам за пять матчей без Эло.

4. Модуль пояснения получает JSON фактов и вероятностей и при необходимости формирует короткий текст. Числа на экране не зависят от пояснения.

5. Интерфейс показывает числа всегда; блок пояснения – только при наличии текста.

Диаграмма последовательности для этого сценария вынесена на отдельный рисунок (рисунок 6).

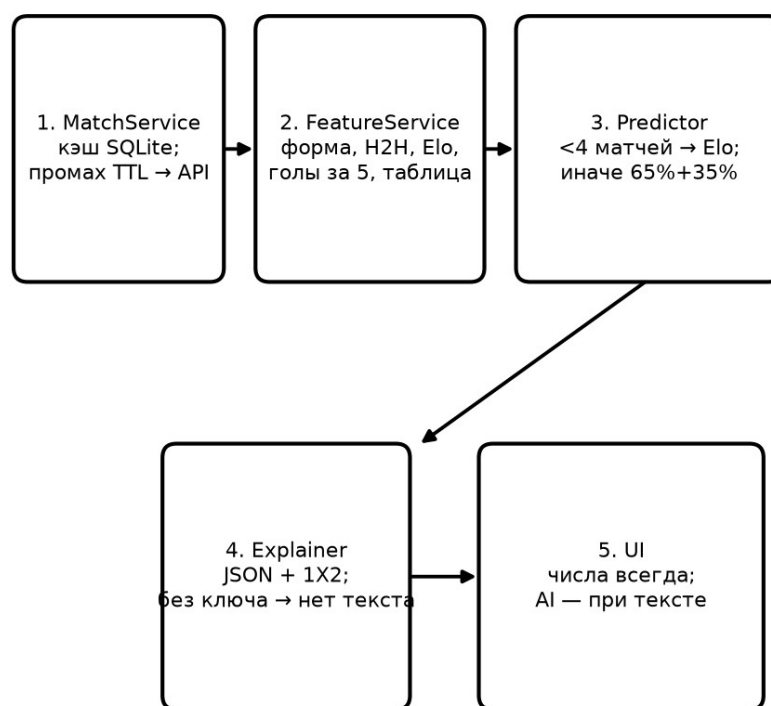


Рисунок 6 – Последовательность расчёта прогноза

Сетевой доступ к football-data.org ограничен тарифом поставщика; ответы кэшируются локально, а в журналы приложения не попадают служебные реквизиты доступа к внешним сервисам [7].

Важное ограничение формулировки: продукт не является веб-приложением в смысле обязательного браузерного клиента. Flet может использовать веб-технологии внутри окна, но сценарий пользователя – запуск настольной программы.

Разделение ответственности между слоями дополнительно защищает от типичных ошибок первой версии. Если бы представление само ходило в HTTP, тесты интерфейса зависели бы от сети. Если бы модель читала SQLite, математический модуль нельзя было бы прогнать на синтетических признаках. Если бы слой AI считал вероятности, исчезла бы граница между объяснением и классификацией. Принятая схема делает каждую из этих ошибок структурно менее вероятной.

С точки зрения развёртывания первая версия ориентирована на поставку собранного настольного приложения: на Windows – исполняемый файл .exe, на macOS – пакет .app. Пользователь запускает готовое окно без установки интерпретатора Python и без команд в терминале. Отдельный сервер баз данных не требуется: файл кэша создаётся локально при первом обращении.

Отказоустойчивость сетевого контура проектируется прагматично. При недоступности API интерфейс должен опираться на ещё свежий кэш и сообщать о невозможности обновить данные, а не «молча» показывать пустой календарь как будто лиг не существует. При превышении лимита запросов клиент обязан замедлиться, а не ретранслировать десятки параллельных вызовов. Эти правила согласованы с документацией поставщика данных [7].

2.2 Информационная модель и модуль прогноза

Информационная модель приложения намеренно узкая. Хранить нужно не «всё о футболе», а тот минимум, без которого нельзя ни показать календарь, ни посчитать прогноз, ни объяснить цифру.

Единица учёта – матч: идентификаторы и названия двух команд, код соревнования, дата и время в UTC, статус, тур, счёт, если он уже есть. Вторая

единица – строка турнирной таблицы: место, число игр, мячи, очки. Третья – снимок признаков на конкретный матч: форма, два рейтинга Эло, средние голы, сводка очных встреч, места в таблице, сколько сыгранных матчей лиги лежит в кэше на дату расчёта. Четвёртая – результат расчёта: три вероятности $1 / X / 2$ и предварительный счёт (два целых, вероятность клетки, два ожидаемых гола).

Данные с сети не идут в интерфейс напрямую. Они оседают в локальном кэше SQLite. Для ещё не сыгранных матчей снимок считается свежим шесть часов, для завершённых и для таблицы – сутки. Это не промышленная СУБД и не «база предприятия»: это кэш, чтобы не превысить лимит бесплатного тарифа (десять запросов в минуту) и чтобы прогноз на одном и том же срезе фактов воспроизводился [7], [15]. Схема основных таблиц кэша показана на рисунке (рисунок 7) и подробно описана в приложении Б.

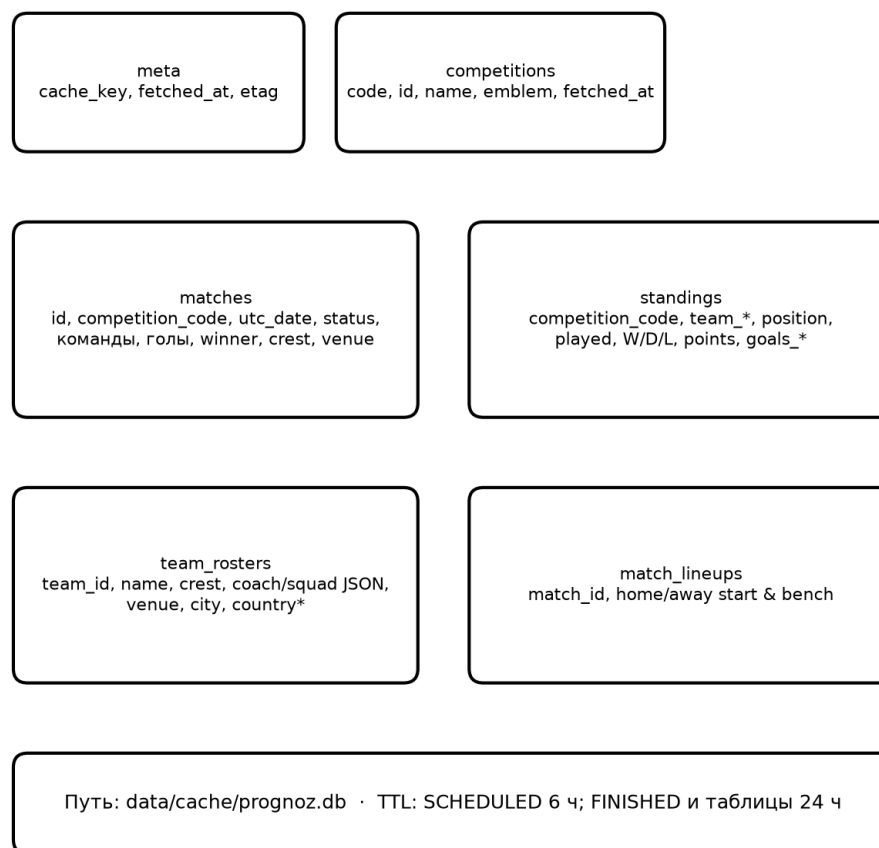


Рисунок 7 – Таблицы локального кэша SQLite

История, из которой считаются Эло и средние голы, обрезается строго: только матчи той же лиги со статусом «завершён» и с датой раньше прогнозируемой встречи. Матч не имеет права видеть свой будущий счёт.

Поля, которые считаются фактом

Для оперативного контура из документации football-data.org v4 в доменную модель матча переносятся: идентификатор, дата UTC, статус, тур, идентификаторы и имена хозяев и гостей, голы полного времени, признак победителя [7]. Для таблицы – место, число игр, победы, ничьи, поражения, мячи, очки. Ожидаемые голы, травмы и иные поля, которых нет в используемом тарифе, в фактах не появляются.

Как из фактов получается 1 / X / 2

Пусть на срез уже посчитаны рейтинги хозяев и гостей и четыре средних по пяти матчам. Если сыгранных матчей лиги меньше четырёх, модуль возвращает только тройку Эло по формулам (2)–(5) главы 1.

Иначе атака и оборона нормируются к уровню лиги 1,35 (ниже 0,4 не опускаются), разница Эло переводится в небольшой множитель, ожидаемые голы ограничиваются отрезком от 0,25 до 4 по формулам (6)–(9). Далее строится сетка (10) и выполняется смесь (11).

Разберём тот же учебный пример, что в главе 1, уже как спецификацию модуля. Исходные оценки:

$$R_{\text{(дом)}} = 1600, R_{\text{(гост)}} = 1580,$$

$$G^{\text{(заб)}}_{\text{(дом)}} = 2,0, G^{\text{(пр)}}_{\text{(дом)}} = 0,8, G^{\text{(заб)}}_{\text{(гост)}} = 2,2, \\ G^{\text{(пр)}}_{\text{(гост)}} = 1,0,$$

число завершённых матчей лиги в кэше равно 20. Тогда

$$a_{\text{(дом)}} = 2,0 / 1,35 \approx 1,4815, d_{\text{(дом)}} = 0,8 / 1,35 \approx 0,5926,$$

$$a_{\text{(гост)}} = 2,2 / 1,35 \approx 1,6296, d_{\text{(гост)}} = 1,0 / 1,35 \approx 0,7407,$$

$$\text{delta} = (1600 + 80 - 1580) / 1000 = 0,10.$$

Интенсивности для ветки исхода:

$$\text{lambda}_{\text{(дом)}} = 1,35 \cdot 1,4815 \cdot 0,7407 \cdot 1,10 \approx 1,63,$$

$$\text{lambda}_{\text{(гост)}} = 1,35 \cdot 1,6296 \cdot 0,5926 \cdot 0,90 \approx 1,17.$$

Отдельно модуль считает тройку Эло и тройку Пуассона, затем смешивает с весами 0,65 и 0,35 и нормирует. На эталонной реализации получены доли 0,4817, 0,2442 и 0,2741 для 1, X и 2 соответственно. Именно эти три числа видит пользователь как ответ на вопрос о предпочтительном исходе.

Как из тех же голов получается предварительный счёт

Для счёта рейтинг Эло сознательно выключается. Ожидаемые голы берутся как среднее атаки одной стороны и обороны другой:

$$\begin{aligned} \lambda_{\text{дом}} &= \frac{G^{\text{заб}}_{\text{дом}} + G^{\text{np}}_{\text{гост}}}{2}, \\ \lambda_{\text{гост}} &= \frac{G^{\text{заб}}_{\text{гост}} + G^{\text{np}}_{\text{дом}}}{2}. \end{aligned} \quad (12)$$

В примере $\lambda_{\text{дом}} = \lambda_{\text{гост}} = 1,50$. На сетке 0...8 выбирается клетка максимальной вероятности. В данном случае это 1:1 с вероятностью около 0,112. Соседние клетки 1:2 и 2:1 имеют меньшие доли. Этот путь не заменяет тройку 1 / X / 2 и не использует округление «вверх хозяевам, вниз гостям».

Вероятность клетки после нормировки хвоста можно записать как

$$P^*(i, j) = \frac{P(X=i, Y=j)}{(\sum_{u=0}^8 \sum_{v=0}^8 P(X=u, Y=v))}, \quad (13)$$

а предварительный счёт – как пару (i^*, j^*) , максимизирующую $P^*(i, j)$ с правилом разрыва связей по близости к $(\lambda_{\text{дом}}, \lambda_{\text{гост}})$.

Инварианты модуля

Модуль обещает:

- на фиксированном срезе фактов результат тот же;
- три вероятности исхода суммируются в единицу;
- предварительный счёт не зависит от Эло и не совпадает с наивным округлением;
- языковая модель не подменяет расчёт.

Модуль не обещает:

- угаданный точный счёт;
- прибыль;
- учёт травм;
- live-счёт без задержки.

На бесплатном тарифе протокол и календарь могут приходить с опозданием – это ограничение источника, его нужно назвать в требованиях и в руководстве пользователя [7].

Роль CSV и обучающих сценариев

Файлы football-data.co.uk с колонками даты, команд, голов и исхода FTR служат длинной историей для ретроспективы и скриптов обучения [8]. Они не подменяют REST-ответ в момент клика по карточке. Такое разделение удерживает информационную модель оперативного контура узкой и проверяемой.

Детализация кэша как информационной структуры

Локальный файл кэша содержит таблицы меток времени, соревнований, матчей, строк турнирной таблицы, заявок и составов. TTL шесть часов для ещё не сыгранных матчей и сутки для завершённых отражает компромисс между актуальностью и лимитом запросов: истечение TTL даёт право снова сходить в сеть, а пока запись свежа, повторный расчёт обязан давать те же числа. Индекс по коду соревнования и дате ускоряет выборку истории; при порче файла его удаляют и наполняют заново из API.

Расширенный разбор смеси и согласование с формулой (1)

При рейтингах 1600 и 1580 тройка Эло составляет примерно 0,4850; 0,2423; 0,2727, тройка Пуассона при $\lambda_{(дом)} \approx 1,63$ и $\lambda_{(гост)} \approx 1,17$ – примерно 0,4799; 0,2453; 0,2748, смесь – 0,4817; 0,2442; 0,2741. Близость троек в этом примере не универсальна: смесь – компромисс первой версии, а не оптимальные веса на всех лигах. Порог «меньше четырёх сыгранных матчей – только Эло» защищает от сетки на почти пустой истории.

Величины K_1 и K_2 из наивной схемы и интенсивности (12) совпадают по способу усреднения. Различие начинается после дроби: наивная схема округляет, модуль показывает моду сетки и её вероятность (для $\lambda = 1,50$ мода 1:1 около 0,112).

2.3 Пользовательский интерфейс

Интерфейс спроектирован как набор экранов одного настольного окна. Базовая навигация включает экраны лиг, календаря матчей и карточки прогноза. Переходы между ними показаны на схеме навигации (рисунок 8). Если снимок экрана в конкретной среде сборки недоступен, текстовое описание ниже остаётся достаточным для понимания сценария; подписи к скриншотам согласованы с манифестом рисунков.

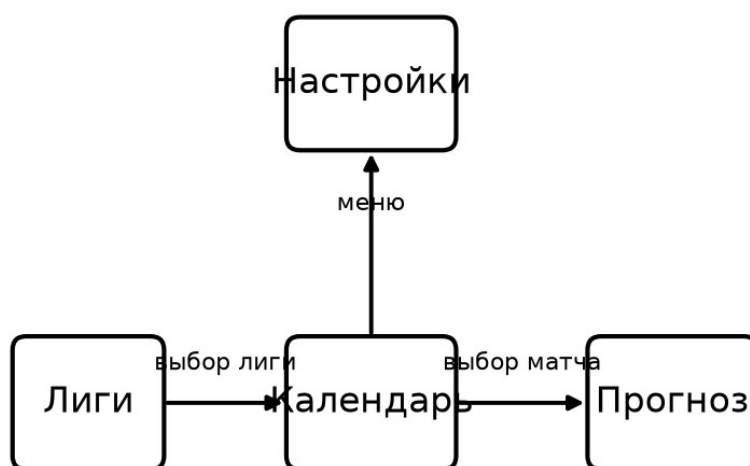


Рисунок 8 – Переходы экранов приложения

Экран лиг. Пользователь видит перечень соревнований бесплатного тарифа. Для каждой позиции отображаются код и название; страна лиги

берётся из локальной карты free-кодов, а не из произвольного поля ответа. Цель экрана – выбрать контекст соревнования и перейти к календарю.

Экран календаря. Список ближайших и релевантных матчей выбранной лиги: дата и время, хозяева, гости, статус. Гербы команд при наличии загружаются с официального CDN crests.football-data.org и кэшируются политикой проекта; живой HTTP гербов из слоя UI запрещён. Выбор строки открывает карточку прогноза.

Экран прогноза матча. Чтение сверху вниз:

- дисклеймер: прогноз статистический, это не совет ставить деньги;
- названия команд и дата;
- широкая полоса 1 / X / 2 с подписями долей – главный ответ на вопрос «кто предпочтительнее»;
- карточка предварительного счёта: крупно клетка, мельче её вероятность и ожидаемые голы, одной фразой сказано, что это средние за пять матчей без Эло и травм;
- четыре поясняющих блока: форма, рейтинг Эло, очные встречи, места в таблице и размер кэша;
- текстовое пояснение при наличии; числа на экране отображаются всегда.

Такой порядок не случаен. Сначала распределение исхода, потом честный «наиболее вероятный счёт» с процентом клетки, потом контекст, потом речь. Речь не имеет права появиться раньше цифр.

Визуальный стиль первой версии – тёмная тема и разделение области списка и области детали (split-экран), чтобы календарь и карточка читались в одном окне на типичном мониторе ноутбука. Тексты интерфейса – на русском языке.

Отдельное внимание уделено состоянию ошибок и пустых данных. Если кэш пуст и сеть недоступна, показывается понятное сообщение. Если матч ещё не имеет достаточной истории в лиге, карточка честно опирается на Эло без

пуассоновской сетки и не маскирует малую выборку декоративной «точностью».

Доступность первой версии ограничена здравым минимумом: контрастный текст на тёмном фоне, читаемые подписи долей, отсутствие критичной информации только в цвете столбца без числовой подписи. Полный аудит доступности не входил в объём работы, но отказ от «голой» визуализации без процентов согласован с вероятностной постановкой задачи.

Сценарий первого запуска:

1. пользователь запускает собранное приложение (.exe или .app);
2. выбирает соревнование в списке лиг;
3. открывает матч и читает полосу 1 / X / 2, затем предварительный счёт и поясняющие блоки;
4. при наличии текстового пояснения сверяет его с числами.

Этот сценарий совпадает с описанием продукта и не включает live-сканер или экран ставок.

Производительность, константы и краевые случаи

Объём кэша одной лиги на горизонте сезона измеряется сотнями матчей; для такого объёма SQLite с индексом по соревнованию и дате достаточен [15]. Узким местом остаётся сеть и лимит 10 запросов в минуту, а не арифметика Пуассона на сетке 0...8. Послойная схема упрощает тесты: модели проверяются на синтетических признаках, клиент API – на локальных образцах, без обращения к живому API.

В первой версии зафиксированы константы, которые нельзя молча менять в тексте и в коде порознь: стартовый рейтинг 1500, К-фактор 20, домашнее преимущество Эло 80 пунктов, коридор ничьей в ветке Эло от 0,18 до 0,32, среднее лиги 1,35, пол относительной атаки и обороны 0,4, клип интенсивностей от 0,25 до 4, потолок сетки 8, веса Пуассона 0,65 и Эло 0,35, порог «только Эло» при числе завершённых матчей меньше 4, окно средних голов 5 матчей, TTL кэша 6 ч и 24 ч.

Если команда впервые появляется в кэше, её рейтинг равен 1500. Если сыгранных матчей меньше пяти, среднее считается по имеющемуся числу игр. Пустая таблица отображается тире, а не нулевыми местами. Пустые гербы не блокируют расчёт. Доменные сущности описывают матч, признаки и вероятности; представление не вызывает Эло напрямую.

Итоговое соответствие «требование – проектное решение»

Таблица 2 – Трассировка ключевых требований к решениям главы 2

Требование	Решение в проекте
Настольное окно	Flet-приложение, поставка <code>.exe` / `.app`</code>
Локальный расчёт 1 / X / 2	Модуль смеси Эло+Пуассон
Предварительный счёт без ceil/floor	Мода сетки по формуле (12)
Лимит 10 запросов/мин	Клиент httpx + TTL-кэш SQLite
LLM не классификатор	Слой ai только поясняет JSON
Дисклеймер	Обязательный блок экрана прогноза
Тесты без живого API	Образцы data/samples

Выводы по главе 2. Выбран стек Python / Flet / httpx / pandas / SQLite с послойной архитектурой, исключающей прямые обращения UI к сети и к файлу кэша. Информационная модель ограничена матчем, таблицей, снимком признаков и результатом расчёта; SQLite играет роль TTL-кэша. Модуль прогноза формализован формулами смеси и предварительного счёта и проверен учебным примером. Интерфейс выстроен вокруг экранов лиг, календаря и прогноза с обязательным дисклеймером и запретом на подмену чисел текстом модели.

3 Реализация и оценка

3.1 Загрузка данных и прогноз

Реализация загрузки данных следует правилу проекта: новый HTTP-источник сначала документируется, затем появляется клиент. Основным оперативным источником – REST API football-data.org версии 4 с базовым адресом <https://api.football-data.org/v4> [7]. Приложение использует, в частности, получение списка соревнований, матчей соревнования с фильтрами сезона и статуса, турнирной таблицы, списка команд и детальной карточки команды. Для отдельного матча допускается запрос детали с площадкой и заявкой; на бесплатном тарифе стартовый состав ещё не сыгранной встречи часто пуст, поэтому в интерфейсе показывают заявку клуба, а не выдуманный состав на игру.

Поля матча, которые переносятся в доменную модель, ограничены документированным набором: идентификатор, дата UTC, статус, тур, идентификаторы и имена команд, голы полного времени, признак победителя. Поля ожидаемых голов и травм отсутствуют и не синтезируются.

Клиент соблюдает лимит бесплатного тарифа: не более десяти запросов в минуту. При коде ответа 429 выполняется ожидание. При коде 403 пользователь получает сообщение о недоступности лиги в тарифе или об отклонении запроса источником. Ответы сохраняются в SQLite-кэше с меткой времени; повторный сетевой запрос выполняется только если записи нет или она старше TTL (шесть часов для расписания, сутки для завершённых результатов и таблиц) [15].

Дополнительный источник – публичные CSV football-data.co.uk с колонками дивизиона, даты, команд, голов и исхода FTR [8]. Парсер поддерживает форматы даты с двузначным и четырёхзначным годом. Скачанные полные сезоны не хранятся в репозитории; в поставку входят

только небольшие образцы в каталоге `samples`. CSV служит обучению и ретроспективе, а не подмене REST в момент открытия карточки.

Цепочка прогноза в реализации совпадает с проектной:

1. сервис матчей обеспечивает актуальный срез кэша;
2. сервис признаков строит снимок только из матчей той же лиги с датой строго раньше прогнозируемой;
3. модуль прогноза выдаёт вероятности и предварительный счёт;
4. модуль пояснения при необходимости формирует короткий текст по JSON без права менять числа;
5. экран отображает результат.

Детерминизм на фиксированных признаках проверяется тестами: повторный вызов даёт ту же тройку вероятностей. При числе завершённых матчей лиги в кэше меньше четырёх Пуассон не запускается, возвращается только Эло. Иначе строится смесь с весами 0,65 и 0,35, как в главах 1 и 2.

Предварительный счёт считается отдельно методом, который использует только средние голы за пять матчей (формула (12)) и моду сетки. Тесты фиксируют, что изменение рейтингов Эло не меняет предварительный счёт, а наивное округление вверх/вниз для интенсивностей 1,2 и 0,8 дало бы 2:0, тогда как мода Пуассона указывает 1:0.

Практическая организация кода следует `src-layout`: исходники пакета лежат в каталоге `src`, тесты – отдельно, документация источников – в `docs`. Такое размещение упрощает установку редактируемого пакета и снижает риск импорта «из корня репозитория» в обход манифеста распределения.

Обработка статусов матча учитывает, что для признаков пригодны завершённые и присуждённые результаты, а для календаря пользователя интересны ещё не сыгранные и уже сыгранные встречи в выбранном окне дат. Статусы берутся из протокола API и не переименовываются произвольно в слое данных.

Синхронизация гербов и флагов вынесена в отдельные служебные сценарии подготовки ресурсов. Живой HTTP за эмблемами из представления

запрещён: либо ресурс уже лежит локально, либо показывается заглушка. Это снижает риск внезапного исчерпания лимита API из-за декоративных загрузок.

3.2 Графический интерфейс и пояснение

Графический интерфейс реализован на Flet как набор представлений: лиги, календарь, карточка матча [6]. Слой представлений не импортирует httpx и не открывает SQLite напрямую: все обращения идут через сервисы. Это упрощает тестирование логики и удерживает архитектуру главы 2.

На экране лиг пользователь выбирает соревнование из двенадцати кодов free-тарифа. На экране календаря отображаются матчи с датой, командами и статусом; при наличии локально подготовленных гербов показываются эмблемы. Карточка матча повторяет порядок чтения, зафиксированный при проектировании: дисклеймер, заголовок, полоса 1 / X / 2, предварительный счёт с вероятностью клетки, блоки формы, Эло, очных встреч и таблицы, затем пояснение.

Пояснение строится по JSON, в который входят имена команд, лига, дата, три вероятности, форма, рейтинги, сводка очных встреч, места, средние голы и предварительный счёт. Промпт запрещает выдумывать травмы, xG и иной счёт. Если пояснение недоступно, блок текста скрывается или показывает служебное сообщение, а числа остаются.

Если файлы скриншотов экранов присутствуют в комплекте рисунков, они вставляются с подписями «экран лиг», «экран календаря», «экран прогноза» (рисунок 9) (рисунок 10) (рисунок 11). Если снимок в среде сборки недоступен, настоящее описание экранов сохраняет смысл главы без отсылки к отсутствующему кадру.

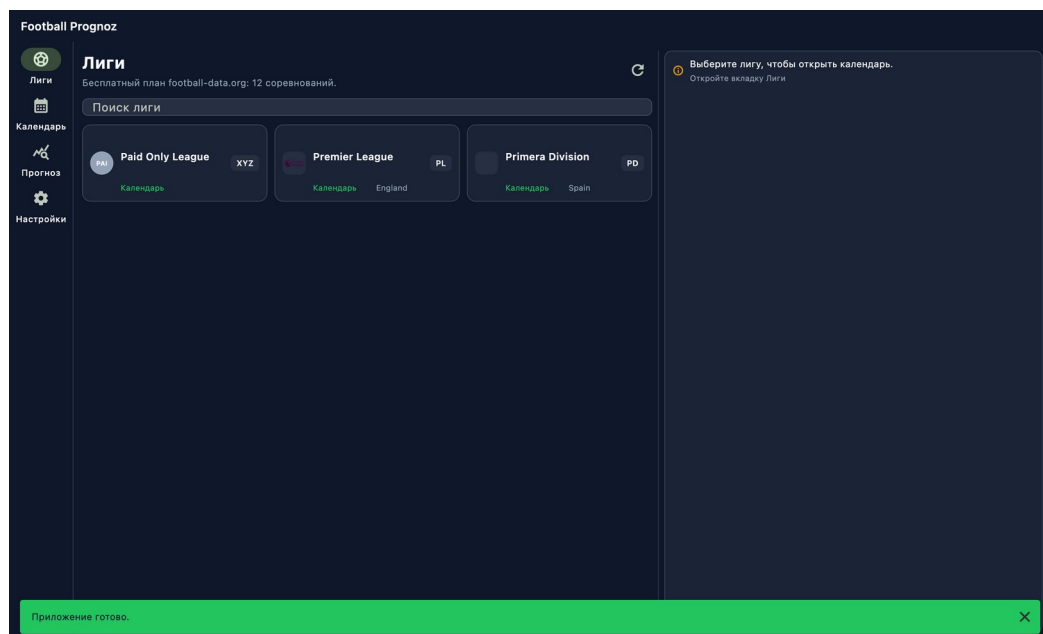


Рисунок 9 – Экран выбора лиг

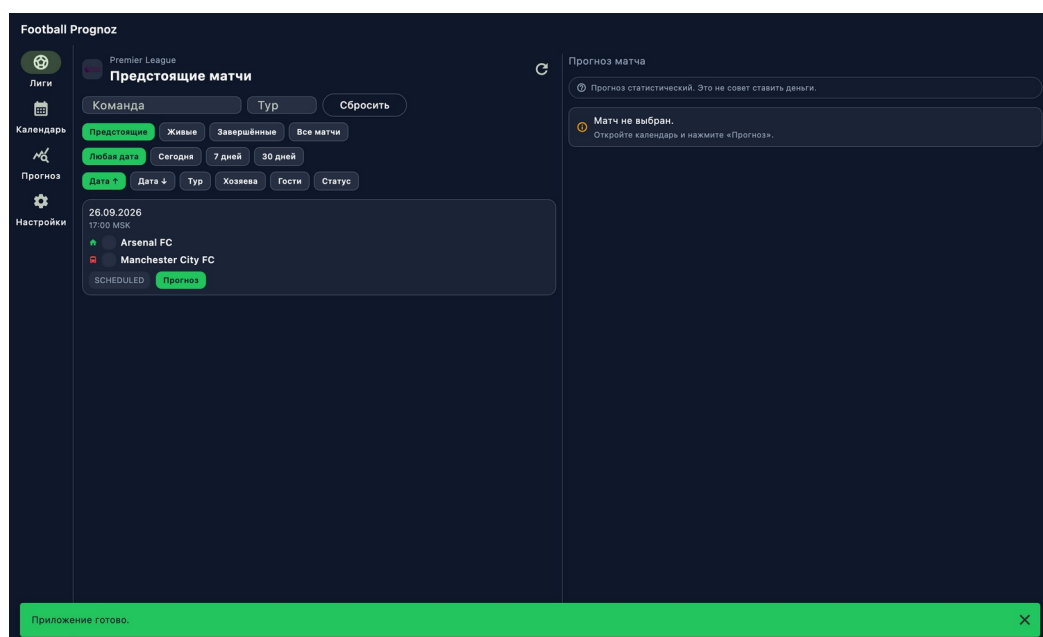


Рисунок 10 – Экран календаря матчей

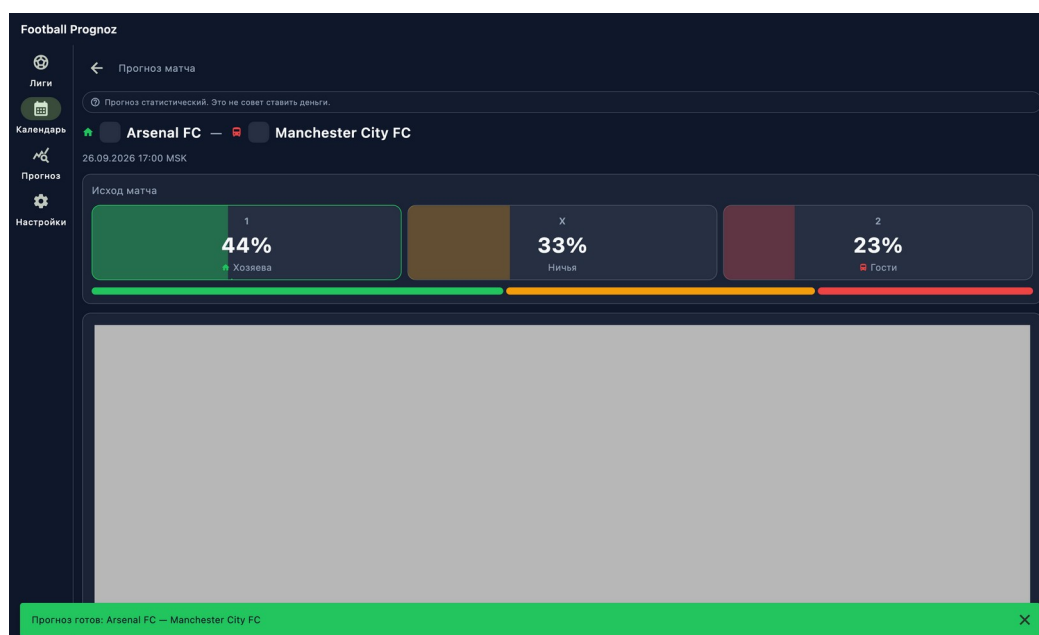


Рисунок 11 – Экран прогноза матча

Текстовое пояснение не является вторым прогнозом. Его задача – помочь человеку связать три процента с видимыми фактами карточки: недавней формой, рейтингами, местами в таблице. Если модель языка нарушит инструкцию и попытается назвать иной счёт, ответственность лежит на промпте и на проверке: в интерфейс всё равно должны попадать числа локального модуля. В первой версии достаточно запрета в инструкции и отсутствия поля «новый счёт» во входном JSON.

3.3 Тестирование и качество прогноза

Модульное тестирование

Автоматические тесты выполняются без обращения к живому API. В качестве входных данных используются локальные образцы. Проверяются, в частности, следующие свойства:

- сумма вероятностей дома, ничьей и гостей равна единице;
- на фиксированных признаках прогноз детерминирован;
- усиление атаки и рейтинга хозяев повышает вероятность победы хозяев;
- сетка Пуассона после обрезания хвоста нормируется;

- предварительный счёт не зависит от Эло;
- предварительный счёт совпадает с модой сетки, а не с округлением вверх/вниз;
- модуль пояснения не подменяет числа выдуманным исходом.

Такой набор не заменяет ретроспективу на полном сезоне, но фиксирует инварианты, без которых нельзя доверять даже учебному примеру.

Метрики вероятностного прогноза

Для оценки вероятностных прогнозов исхода 1 / X / 2 в литературе и практике используют логарифмические потери и оценку Бриера [3]. Пусть для матча модель выдала вектор (p_1, p_X, p_2) , а факт закодирован одним из единичных векторов y in $(1,0,0), (0,1,0), (0,0,1)$. Оценка Бриера для одного матча:

$$B = (p_1 - y_1)^2 + (p_X - y_X)^2 + (p_2 - y_2)^2. \quad (14)$$

Средняя оценка Бриера по выборке из N матчей:

$$\overline{B} = (1/N) \sum_{n=1}^N B_n. \quad (15)$$

Чем меньше $\overline{B}$, тем лучше. Для равномерного прогноза $(1/3, 1/3, 1/3)$ значение B на любом исходе равно $2/3 \approx 0,6667$.

Логарифмические потери для одного матча:

$$L = -\ln p_*(*), \quad (16)$$

где $p_*(*)$ – вероятность, которую модель присвоила фактически состоявшемуся исходу (с нижней отсечкой для численной устойчивости). Средние потери по выборке интерпретируются так же: меньше – лучше. Равномерный прогноз даёт $L = -\ln(1/3) \approx 1,0986$.

Дополнительно строят матрицу соответствия между ярлыком наибольшей доли (1, X или 2) и фактическим исходом. Эта матрица наглядна,

но грубее вероятностных метрик: она выбрасывает информацию о том, была ли доля фаворита 0,40 или 0,70.

Калибровка («если модель часто ставила около 60 % на хозяев, как часто хозяева в самом деле выигрывали») требует достаточного числа матчей в каждом коридоре вероятности. На крошечной выборке калибровку не заявляют.

Наивная эвристика пяти матчей переводится в исход так: по формуле (1) получают K_1 и K_2 , округляют вверх и вниз, сравнивают целые и получают ярлык 1, X или 2. Такой ярлык можно сравнивать с фактом и с ярлыком смеси Эло+Пуассон, но вероятностные метрики к нему применимы только после искусственного сопоставления (например, единичного вектора), что делает сравнение с мягкими вероятностями смеси несимметричным. Поэтому в учебном примере ниже наивная схема фигурирует как классификатор ярлыка, а смесь – и как вероятностный прогноз, и как ярлык argmax .

Учебный пример на локальном образце CSV

В репозитории находится файл `data/samples/sample_results.csv` из шести завершённых строк чемпионата Англии (колонки Div, Date, HomeTeam, AwayTeam, FTHG, FTAG, FTR). Это учебный фрагмент, а не полный сезон. Полная ретроспектива качества на чемпионате в настоящей работе не заявляется.

На этих шести матчах выполнен упрощённый проход «вперёд по времени»: рейтинги Эло и окна последних матчей обновляются только после прогноза очередной встречи; подглядывания в будущий счёт нет. Чтобы одновременно посчитать и Эло, и средние голы для обеих команд, у каждой стороны должна быть хотя бы одна завершённая игра в прошлом относительно даты матча. В данном образце такому условию удовлетворяет ровно одна встреча: Chelsea – Man City от 21.09.2024 с фактическим исходом «победа гостей» (FTR = A, счёт 0:2).

Для этой единственной точки получены следующие числа (округление до четырёх знаков):

Таблица 3 – Учебный пример вероятностных метрик на одном матче образца CSV

Метод	p(1)	p(X)	p(2)	Ярлык	Brier	Log-loss
Смесь Эло+Пуассон	0,24 68	0,21 61	0,53 71	2	0,32 19	0,62 16
Только Эло	0,45 51	0,24 86	0,29 63	1	0,76 42	1,21 65
Равномерный (1/3)	0,33 33	0,33 33	0,33 33	–	0,66 67	1,09 86
Эвристика ceil/floor	–	–	–	2	–	–

Примечание – Вероятности Эло и смеси получены в упрощённом проходе по шести строкам образца с обновлением рейтингов только после прогноза. Эвристика на этой точке дала счёт после округления 1:2 и ярлык 2. На выборке из одного матча доля верных ярлыков не интерпретируется как качество сезона.

Таблица 4 – Сводка по единственной оцениваемой точке образца

Показатель	Значение
Число строк в sample_results.csv	6
Число матчей с прогнозом смеси	1
Фактический исход	2
Верный ярлык смеси	да
Верный ярлык только Эло	нет
Верный ярлык эвристики	да
Brier смеси	0,3219
Brier только Эло	0,7642
Brier равномерной базы	0,6667
Log-loss смеси	0,6216
Log-loss только Эло	1,2165
Log-loss равномерной базы	1,0986

Интерпретация. На одном матче смесь по вероятностным метрикам лучше и равномерной базы, и ветки «только Эло» (ниже Brier и log-loss), потому что существенная доля вероятности была отдана фактическому исходу «2» ($p_2 \approx 0,54$). Ярлык argmax смеси и ярлык эвристики совпали с фактом, тогда как «только Эло» указал на хозяев. Эвристика не дала калиброванных вероятностей, поэтому по Brier и log-loss с мягкой смесью напрямую не сравнивается. Переносить эти наблюдения на «точность чемпионата» нельзя: объём образца слишком мал.

Таблица 5 – Матрица ярлыков для единственной оцениваемой точки (смесь)

–	Факт 1	Факт X	Факт 2
Прогноз 1	0	0	0
Прогноз X	0	0	0
Прогноз 2	0	0	1

Таблица 6 – Матрица ярлыков для единственной оцениваемой точки (только Эло)

–	Факт 1	Факт X	Факт 2
Прогноз 1	0	0	1
Прогноз X	0	0	0
Прогноз 2	0	0	0

Таблицы 5 и 6 демонстрируют формат отчёта, который следует строить на полной ретроспективе. На одном наблюдении они лишь фиксируют, что смесь попала в клетку «прогноз 2 / факт 2», а ветка Эло – в клетку «прогноз 1 / факт 2».

Для сравнения методов на большой выборке имеет смысл одновременно публиковать:

- средний Brier и средний log-loss;
- матрицу ярлыков argmax ;

- долю верных ярлыков только как вспомогательный показатель;
- отдельно – долю верных ярлыков наивной эвристики;
- при достаточном объёме – таблицу калибровки по коридорам вероятности фаворита.

Без достаточного N пункты калибровки и «точности сезона» в текст не вносят.

Учебный числовой пример модуля без ретроспективы

Независимо от CSV полезен эталонный срез признаков из модульных тестов (рейтинги 1600 и 1580, средние голы 2,0 / 0,8 и 2,2 / 1,0, двадцать матчей в кэше). На нём смесь даёт

$$P(1) \approx 0,4817, P(X) \approx 0,2442, P(2) \approx 0,2741,$$

а предварительный счёт – 1:1 с вероятностью клетки около 0,112 при $\lambda = 1,50$ для обеих сторон. Эти числа воспроизводимы командой тестов и служат сверкой формул глав 1–2 с кодом. Исход матча для этого среза в тестах не фиксируется, поэтому Brier и log-loss на нём не считаются: нет факта y .

Что остаётся за рамками заявленного качества

Не заявляются и не приводятся выдуманные показатели вида «точность 58 % на сезоне Premier League» или «Brier 0,XX на всех лигах free-тарифа», если соответствующий прогон на полной выборке в работе не выполнен. Скрипт обучения на CSV может быть развит до полной ретроспективы в последующих работах; в текущей пояснительной записке честная граница такова: инварианты покрыты тестами, метрики определены формулами (14)–(16), на образце из шести строк посчитана одна учебная точка, эталонный срез признаков сверен с реализацией.

Таблица 7 – Что измерено в работе и что намеренно не заявлено

Вопрос	Статус в настоящей записке
Сумма вероятностей равна 1	Проверено тестами
Детерминизм на фиксированных признаках	Проверено тестами
Предварительный счёт независим от Эло	Проверено тестами
Эталонный срез: тройка 1 / X / 2 и мода 1:1	Вычислено и сверено с кодом
Brier / log-loss на sample_results.csv	Учебный пример, N = 1
Калибровка по коридорам вероятности	Не заявлена
Точность на полном чемпионате	Не заявлена
Прибыль относительно коэффициентов	Не рассматривается

Такая таблица нужна, чтобы читатель не принял учебный пример за обещание промышленного качества.

План полной ретроспективы и сводка учебных чисел

Для последующей работы рекомендуется хронологический проход по сезонному CSV football-data.co.uk [8] с накоплением Brier, log-loss, матриц ярлыков и калибровки по коридорам вероятности. До выполнения этого протокола в записке допустимы лишь определения метрик, эталонный срез и учебная точка на шести строках образца.

Для удобства защиты ниже собраны только величины, реально полученные в этой работе.

Таблица 8 – Сводка вычисленных учебных чисел

Источник	Величина	Значение
Эталонный срез признаков	P(1), P(X), P(2)	0,4817; 0,2442; 0,2741
Эталонный срез признаков	Предварительный счёт	1:1 ($\approx 0,112$)
Эталонный срез признаков	λ предварительного счёта	1,50; 1,50
sample_results.csv, N	Brier смеси	0,3219

= 1		
sample_results.csv, N = 1	Brier Эло	0,7642
sample_results.csv, N = 1	Brier равномерной базы	0,6667
sample_results.csv, N = 1	Log-loss смеси	0,6216
sample_results.csv, N = 1	Log-loss Эло	1,2165
sample_results.csv, N = 1	Log-loss равномерной базы	1,0986

Все строки таблицы 8 либо воспроизводятся модульными тестами, либо воспроизводятся описанным проходом по шести строкам CSV. Строк вида «точность чемпионата = ...» в таблице нет намеренно.

Выводы по главе 3. Реализованы загрузка данных через документированный REST и CSV, TTL-кэш и цепочка прогноза с детерминированной смесью Эло и Пуассона. Интерфейс показывает числа независимо от наличия языковой модели и сохраняет дисклеймер. Тесты закрепляют инварианты. Качество описано корректными метриками; численные значения даны только как учебный пример на локальном образце и на эталонном срезе признаков, без претензии на ретроспективу чемпионата.

Заключение

В ходе выпускной квалификационной работы спроектировано и реализовано настольное приложение статистического прогнозирования исходов футбольных матчей. Цель достигнута: вероятности $1 / X / 2$ и предварительный точный счёт считаются локальной моделью, а языковая модель только поясняет уже полученные числа и факты карточки.

Решены поставленные задачи. Проанализированы особенности прогноза и выбраны методы: рейтинг Эло, независимый Пуассон голов по мотивам модели Махера и смесь двух оценок с весами 0,65 и 0,35. Наивная эвристика «пять матчей и округление» разобрана и оставлена базовой линией сравнения, а не движком экрана. Сопоставлены существующие решения спортивной аналитики и зафиксирована ниша прозрачного десктопа. Сформулированы требования, включая дисклеймер, поставку собранного приложения для Windows и macOS и лимит десяти запросов в минуту. Спроектирована послойная архитектура `ui` → `services` → `data / models / ai` на стеке Python, Flet, httpx, pandas и SQLite. Реализованы загрузка данных football-data.org v4 и CSV football-data.co.uk, TTL-кэш, модуль прогноза и экраны лиг, календаря и прогноза. Проведены модульные тесты инвариантов; метрики Brier и log-loss определены формулами и проиллюстрированы учебным примером на локальном образце без претензии на качество полного чемпионата.

Практическим результатом является работоспособное приложение и комплект приложений А–Д к пояснительной записке. Методическим результатом – согласованная цепочка «факт – оценка – вывод» и явный отказ подменять локальный расчёт ответом нейросети или округлением средних голов.

Ограничения первой версии сохраняют силу: двенадцать лиг free-тарифа, возможная задержка протокола, отсутствие xG и травм в используемых полях, упрощённый независимый Пуассон без поправки Диксона–Коулза, малый объём учебного прогона метрик.

Направления развития:

- выполнить полную ретроспективу Brier и log-loss на сезонных CSV с честными таблицами калибровки;
- оценить веса смеси и константу среднего лиги на данных, а не только зафиксировать значения первой версии;
- при необходимости реализовать поправку Диксона–Коулза как отдельный сравниваемый вариант;
- расширить покрытие данных только после документирования нового источника;
- улучшить упаковку настольного дистрибутива для пользователя без среды разработки.

В приложениях собраны материалы, которые дополняют основную часть, не раздувая её листингами: техническое задание (приложение А), описание TTL-кэша (приложение Б), руководство пользователя (приложение В), ключевые фрагменты кода (приложение Г) и числовые примеры прогнозов (приложение Д). Основной текст при этом сохраняет связное изложение методов, проектирования и оценки.

Итог можно сформулировать одной фразой: построено настольное статистическое приложение с воспроизводимой локальной моделью $1 / X / 2$, честным предварительным счётом и запретом подменять расчёт языковой моделью или наивным округлением.

Работа выполнена в рамках заявленной темы настольного статистического приложения и не подменяет её формулировками о тотализаторе или системе ставок.

Прогноз приложения остаётся статистической оценкой и не является советом ставить деньги.

Список используемой литературы и используемых источников

1. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. М. : Издательство стандартов, 1980.
2. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. М. : Стандартиформ, 2020.
3. Brier G. W. Verification of forecasts expressed in terms of probability // Monthly Weather Review. 1950. Vol. 78, No. 1. P. 1–3.
4. Dixon M. J., Coles S. G. Modelling Association Football Scores and Inefficiencies in the Football Betting Market // Journal of the Royal Statistical Society: Series C (Applied Statistics). 1997. Vol. 46, No. 2. P. 265–280.
5. Elo A. E. The Rating of Chessplayers, Past and Present. New York : Arco Publishing, 1978. 202 p.
6. Flet documentation [Электронный ресурс]. URL: <https://flet.dev/docs/> (дата обращения: 22.09.2026).
7. Football-Data.org. API Documentation [Электронный ресурс]. URL: <https://www.football-data.org/documentation/api> (дата обращения: 22.09.2026).
8. Football-Data.co.uk. Historical Football Results and Betting Odds Data [Электронный ресурс]. URL: <https://www.football-data.co.uk/data.php> (дата обращения: 22.09.2026).
9. Hvattum L. M., Arntzen H. Using ELO ratings for match result prediction in association football // International Journal of Forecasting. 2010. Vol. 26, No. 3. P. 460–470.
10. HTTPX documentation [Электронный ресурс]. URL: <https://www.python-httpx.org/> (дата обращения: 22.09.2026).
11. Maher M. J. Modelling association football scores // Statistica Neerlandica. 1982. Vol. 36, No. 3. P. 109–118.

12. pandas documentation [Электронный ресурс]. URL: <https://pandas.pydata.org/docs/> (дата обращения: 22.09.2026).
13. Python Software Foundation. Python 3 Documentation [Электронный ресурс]. URL: <https://docs.python.org/3/> (дата обращения: 22.09.2026).
14. scikit-learn User Guide [Электронный ресурс]. URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 22.09.2026).
15. SQLite Documentation [Электронный ресурс]. URL: <https://www.sqlite.org/docs.html> (дата обращения: 22.09.2026).

Приложение А

Техническое задание на разработку настольного приложения статистического прогнозирования исходов футбольных матчей

Настоящее приложение содержит сжатое техническое задание, оформленное с учётом требований к содержанию технического задания на программное средство и автоматизированную систему [1], [2].

А.1 Наименование и область применения

Наименование: настольное приложение Football Prognoz.

Область применения: персональный компьютер пользователя под управлением Windows 10/11 (64-bit) или macOS 12 и новее; учебный и аналитический просмотр вероятностей исхода футбольных матчей по открытым данным.

А.2 Основание для разработки

Основание – задание на выпускную квалификационную работу по теме «Разработка настольного приложения статистического прогнозирования исходов футбольных матчей».

А.3 Назначение

Приложение предназначено для:

- выбора футбольного соревнования из набора бесплатного тарифа football-data.org;
- просмотра календаря матчей;
- расчёта локальной моделью вероятностей исхода 1 / X / 2 и предварительного точного счёта;
- отображения поясняющих агрегатов (форма, рейтинги Эло, очные встречи, таблица);
- опционального текстового пояснения уже вычисленных вероятностей языковой моделью.

Приложение не предназначено для приёма ставок, расчёта прибыли и выдачи советов ставить деньги.

A.4 Требования к программе

A.4.1 Функциональные требования

1. Система должна загружать список соревнований free-тарифа (коды PL, PD, SA, BL1, FL1, DED, PPL, ELC, BSA, CL, WC, EC).
2. Система должна отображать матчи выбранного соревнования с датой, командами и статусом.
3. Система должна рассчитывать вероятности 1 / X / 2 смесью Эло и независимого Пуассона (при малой выборке – только Эло).
4. Система должна рассчитывать предварительный счёт как моду сетки Пуассона по средним голам за пять матчей без Эло и без округления вверх/вниз как единственного ответа.
5. Система должна показывать дисклеймер о статистическом характере прогноза.
6. Система должна кэшировать ответы API в локальном файле SQLite с TTL 6 ч / 24 ч.
7. Система должна соблюдать лимит не более 10 запросов в минуту.
8. Языковая модель не должна выбирать исход; допускаются только пояснения по готовому JSON.

A.4.2 Требования к надёжности

Расчёт на фиксированном срезе фактов должен быть детерминированным. Сумма трёх вероятностей исхода должна равняться 1. При отсутствии сети допускается работа по свежему кэшу.

A.4.3 Условия эксплуатации

Пользователь запускает собранное настольное приложение на Windows 10/11 (64-bit) или macOS 12 и новее. Доступ в сеть нужен для первичной загрузки календаря и результатов.

A.4.4 Требования к составу и параметрам технических средств

Персональный компьютер под управлением Windows 10/11 (64-bit) или macOS 12 и новее, доступ в сеть для первичной загрузки данных.

А.4.5 Требования к информационной и программной совместимости

Язык реализации Python 3.11+, графическая библиотека Flet, клиент httpx, библиотека pandas, встроенный движок SQLite, при обучении – scikit-learn. Для пользователя программа поставляется как .exe (Windows) или .app (macOS).

А.5 Стадии разработки

1. Анализ предметной области и требований.
2. Проектирование архитектуры и информационной модели.
3. Реализация загрузки данных, кэша, модели и интерфейса.
4. Тестирование и оформление пояснительной записки.

А.6 Порядок контроля и приёмки

Приёмка первой версии: запуск собранного окна на целевой платформе; загрузка лиг и матчей; воспроизводимый прогноз на фиксированных признаках; наличие дисклеймера; прохождение модульных тестов без живого API.

Приложение Б

Описание локального кэша SQLite

Локальный файл кэша (типичный путь data/cache/prognosz.db) не является промышленной СУБД. Это встраиваемое хранилище ответов API с метками времени и политикой TTL [15].

Б.1 Назначение таблиц

Таблица Б.1 – Основные таблицы кэша

Таблица	Назначение
meta	Ключ кэша, время получения, необязательный ETag
competitions	Код и название соревнования, эмблема, время получения
matches	Матчи лиги: дата, статус, команды, счёт, гербы, площадка
standings	Строки турнирной таблицы
team_rosters	Заявка клуба (тренер и состав в JSON), площадка, страна
match_lineups	Составы конкретной встречи при наличии в API

Б.2 Политика времени жизни

Таблица Б.2 – TTL по типам данных

Тип данных	TTL
Расписание / ещё не сыгранные матчи	6 ч
Завершённые матчи	24 ч
Турнирные таблицы	24 ч
Деталь матча (заявка/lineup)	6 ч

Повторный запрос к сети выполняется только при отсутствии записи или при истечении TTL, с соблюдением лимита 10 запросов в минуту [7].

Б.3 Логическая схема (текстом)

- `meta(cache\_key PK, fetched\_at, etag)`;
- `competitions(code PK, id, name, emblem, fetched\_at)`;
- `matches(id PK, competition\_code, utc\_date, status, matchday, home\_id, home\_name, away\_id, away\_name, home\_goals, away\_goals, winner, home\_crest, away\_crest, venue, fetched\_at)`;
- индекс `idx\_matches\_comp\_date(competition\_code, utc\_date)`;
- `standings(competition\_code, team\_id, ..., PRIMARY KEY(competition\_code, team\_id))`;
- `team\_rosters(team\_id PK, ...)`;
- `match\_lineups(match\_id PK, ...)`.

Рисунок логической схемы кэша в основном тексте согласован с файлом [figures/drawio/sqlite-cache.png](#) (рисунок 7).

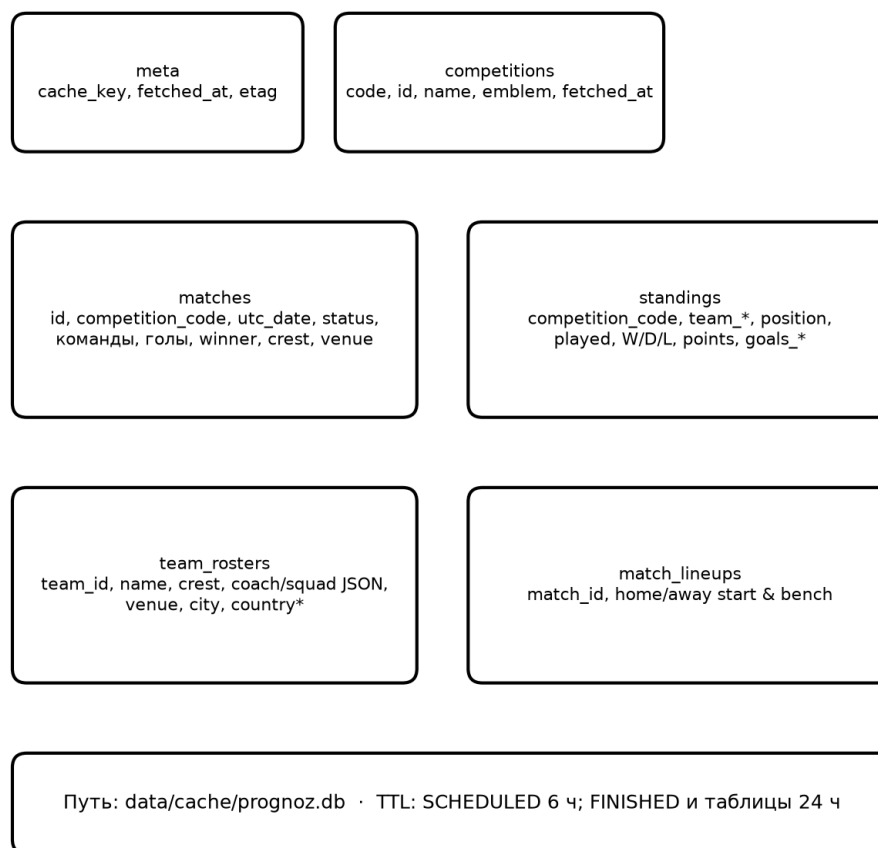


Рисунок 7 – Таблицы локального кэша SQLite

Б.4 Ограничения

Кэш не моделирует пользователей, платежи и журнал ставок. Резервное копирование корпоративного уровня не требуется: при повреждении файл удаляют и наполняют заново из API.

Приложение В

Руководство пользователя

В.1 Назначение

Приложение Football Prognosz показывает статистические вероятности исхода футбольного матча 1 / X / 2 и предварительный точный счёт. Это не букмекер и не совет ставить деньги.

В.2 Системные требования

- Windows 10/11 (64-bit) или macOS 12 и новее;
- доступ в интернет для первичной загрузки календаря и результатов.

В.3 Установка и запуск

Программа поставляется как собранное настольное приложение. Пользователю не требуется устанавливать интерпретатор Python или выполнять команды в терминале.

1. На Windows запустите файл с расширением .exe.
2. На macOS откройте приложение с расширением .app.
3. Дождитесь появления главного окна и перейдите к списку лиг.

В.4 Основные экраны

Лиги. Выберите соревнование из списка free-тарифа.

Календарь. Просмотрите матчи: дата, хозяева, гости, статус. Откройте нужную встречу.

Прогноз. Прочитайте дисклеймер, затем полосу вероятностей 1 / X / 2, затем карточку предварительного счёта с процентом клетки и ожидаемыми голами. Ниже – форма, рейтинги Эло, очные встречи, места в таблице. При наличии текстового пояснения оно появляется после чисел и не заменяет их.

В.5 Ограничения данных

На бесплатном тарифе football-data.org доступны двенадцать соревнований и не более десяти запросов в минуту. Счёт и расписание могут приходить с задержкой. Поля xG и травм отсутствуют и не отображаются как факты.

В.6 Типичные сообщения

- Нет сети – проверьте подключение; при наличии свежего кэша календарь может остаться доступным;
- Ошибка 403 – лига недоступна в используемом тарифе или источник временно отклонил запрос;
- Ошибка 429 – превышен лимит запросов; подождите и повторите позже;
- Пояснение недоступно – числа прогноза всё равно отображаются.

Приложение Г

Листинги ключевых фрагментов

Ниже приведены сокращённые фрагменты для иллюстрации идей реализации. Полные исходные тексты находятся в репозитории проекта. Нумерация листингов локальна для приложения Г.

Листинг Г.1 – Вероятность клетки независимого Пуассона (идея)

```
def poisson_pmf(k: int, lam: float) -> float:
    if lam <= 0:
        return 1.0 if k == 0 else 0.0
    return math.exp(-lam) * lam**k / math.factorial(k)
```

Листинг Г.2 – Смесь Пуассона и Эло (идея)

```
poisson = poisson_1x2(lam_home, lam_away)
elo = Probabilities(*elo_1x2(home_elo, away_elo)).normalized()
return blend(poisson, elo, weight_a=0.65)
```

Листинг Г.3 – Предварительный счёт без Эло (идея)

```
lam_home = (home_gf + away_ga) / 2.0
lam_away = (away_gf + home_ga) / 2.0
return poisson_mode(lam_home, lam_away) # не ceil/floor
```

Листинг Г.4 – Ожидание Эло с домашним преимуществом (идея)

```
exp_home = expected_score(home_rating + 80.0, away_rating)
home_rating += 20.0 * (actual_home - exp_home)
```

Листинг Г.5 – Проверка свежести кэша (идея)

```
TTL_SCHEDULED_HOURS = 6  
TTL_FINISHED_HOURS = 24
```

```
def is_fresh(cache_key: str, ttl_hours: float) -> bool:  
    fetched = fetched_at(cache_key)  
    return fetched is not None and (utcnow() - fetched) < timedelta(hours=ttl_hours)
```

Листинг Г.6 – Граница пояснения LLM (идея)

Вход: JSON фактов + вероятности + предварительный счёт.
Запрещено: угадывать иной счёт, выдумывать травмы и xG.
Выход: 2–4 предложения пояснения либо отсутствие текста, если пояснение недоступно.

Имена модулей и полные пути см. в инженерной документации репозитория (docs/FORECAST.md, docs/ARCHITECTURE.md).

Приложение Д

Примеры прогнозов

Примеры ниже иллюстрируют различие факта, оценки и вывода. Они не являются советом ставить деньги.

Д.1 Эталонный срез признаков (модульные тесты)

Исходные оценки:

- рейтинги Эло: хозяева 1600, гости 1580;
- средние голы за пять матчей: хозяева 2,0 забитых и 0,8 пропущенных; гости 2,2 и 1,0;
- число завершённых матчей лиги в кэше: 20.

Вывод модуля (сверено с реализацией):

- вероятности $1 / X / 2 \approx 0,4817 / 0,2442 / 0,2741$;
- предварительный счёт: 1:1;
- вероятность клетки $\approx 0,112$;
- ожидаемые голы для предварительного счёта: 1,50 : 1,50.

Для сравнения: наивная эвристика с округлением тех же K_1 и K_2 дала бы 2:1.

Д.2 Учебная точка на sample_results.csv

Матч Chelsea – Man City, дата 21.09.2024, факт: 0:2, исход 2.

Упрощённый проход вперёд по шести строкам образца (обновление рейтингов только после прогноза) дал:

- смесь: $p \approx 0,2468 / 0,2161 / 0,5371$, ярлык 2;
- только Эло: $p \approx 0,4551 / 0,2486 / 0,2963$, ярлык 1;
- эвристика ceil/floor: ярлык 2;
- Brier смеси 0,3219; Brier Эло 0,7642; Brier равномерной базы 0,6667;
- log-loss смеси 0,6216; log-loss Эло 1,2165; log-loss равномерной базы 1,0986.

Это учебный пример на $N = 1$, не качество чемпионата.

Д.3 Как читать карточку пользователю

1. Сначала три процента исхода.
2. Затем предварительный счёт вместе с процентом клетки.
3. Затем форма, Эло, очные встречи и таблица как контекст.
4. Затем текст пояснения, если он есть.
5. Дисклеймер действует на все числа экрана.